2026.03.24

Anthropic Engineering Blog

# 장시간 실행 앱 개발을 위한 하네스 설계

Harness Design for Long-Running Application Development

Prithvi Rajasekaran · Anthropic Labs

GAN-Inspired | Multi-Agent | Generator-Evaluator

CORE QUESTION

# 고품질 프론트엔드와 완전 자율 개발, 두 마리 토끼를 잡는 법

GAN에서 영감을 받은 Generator-Evaluator 멀티 에이전트 시스템으로 주관적 판단을 구체적이고 채점 가능한 기준으로 변환

01

### 단순 구현의 한계

컨텍스트 윈도우 열화와 자기평가 편향이 장시간 태스크에서 품질을 저하

02

### Generator-Evaluator 분리

생성과 평가를 분리하여 객관적 품질 향상의 강력한 레버 확보

03

### 하네스의 진화

모델 발전에 따라 불필요한 스캐폴딩을 제거하고 새로운 조합 탐색

01

# 핵심 문제점

왜 단순한 구현은 실패하는가

WHY NAIVE FAILS

# 두 가지 치명적 문제

**A**

### 컨텍스트 윈도우 열화

Context Window Degradation

- ✓ 긴 작업에서 컨텍스트가 채워지면 모델 일관성 저하
- ✓ "컨텍스트 불안" — 한계에 가까워지면 조기 마무리 경향
- → **해결:** 압축보다 완전 리셋이 더 효과적

**B**

### 자기평가 편향

Self-Evaluation Bias

- ✓ 에이전트가 자기 결과물을 자신 있게 칭찬 (품질 무관)
- ✓ 디자인 같은 주관적 태스크에서 특히 치명적
- → **해결:** 생성 에이전트와 평가 에이전트를 분리

02

# 프론트엔드 디자인 품질 평가

주관적 품질을 채점 가능하게 만드는 방법

GRADING CRITERIA

# 4가지 디자인 평가 기준

01

### 디자인 품질 (Design Quality)

부분의 집합이 아닌 하나의 일관된 전체로 느껴지는가? 색상, 타이포그래피, 레이아웃, 이미지가 결합되어 고유한 분위기와 정체성을 만드는가?

02

### 독창성 (Originality)

템플릿 기본값이나 AI 패턴(보라색 그라데이션 + 흰 카드)이 아닌, 인간 디자이너가 인식할 수 있는 의도적 선택의 증거가 있는가?

03

### 완성도 (Craft)

타이포그래피 위계, 간격 일관성, 색상 조화, 대비율 등 기술적 실행 수준. 창의성이 아닌 역량 검증

04

### 기능성 (Functionality)

미적 요소와 독립적으로 사용자가 인터페이스를 이해하고, 주요 액션을 찾고, 직관적으로 완료할 수 있는가?

SYSTEM ARCHITECTURE

# Generator-Evaluator 반복 시스템

Claude Agent SDK 기반 생성 → Playwright MCP 기반 평가 → 5~15회 반복

**Generator**
HTML/CSS/JS
프론트엔드 생성

→

**Live Page**
실제 브라우저
렌더링

→

**Evaluator**
네비게이션 + 스크린샷
기준별 채점

→

**전략 결정**
개선 vs
완전 피벗

| 5~15 | 4h | 4 | 2 |
|---|---|---|---|
| 반복 횟수/생성 | 전체 실행 시간 | 평가 기준 | 전략 옵션 |

> *"평가 기준의 문구가 Generator를 예상치 못한 방향으로 이끌었다. 'museum quality' 같은 표현이 특정 시각적 수렴을 유도했다."*

CASE STUDY

# 네덜란드 미술관 웹사이트 — 10번째 반복의 돌파

### 반복 1~9회: 점진적 개선

1. 기본적 레이아웃 + 색상 체계 수립
5. 타이포그래피 위계와 간격 정리
9. **세련되었지만 전형적인 디자인** — 컨벤셔널한 갤러리 레이아웃

### 반복 10회: 완전한 재구상

- **CSS 원근법** 으로 렌더링된 3D 방
- **체커보드 바닥** 위의 공간적 경험
- 벽에 **자유롭게 배치된 작품**
- 갤러리가 아닌 **"공간적 경험"** 으로 탈바꿈

✓ 첫 번째 반복 출력도 기존 기준선을 이미 초과 — 프롬프팅 자체가 모델을 제네릭 디자인에서 이탈하도록 유도

03

# 풀스택 코딩으로 확장

3-에이전트 아키텍처와 스프린트 계약

THREE-AGENT ARCHITECTURE

# 3-에이전트 아키텍처

에이전트 간 통신은 파일 기반 — 명세 충실성 유지 + 구현 자유도 확보

P

## Planner

입력: 1~4문장

단순 프롬프트를 종합 제품 명세서로 확장. 세부 구현이 아닌 **결과물과 기술 방향** 중심. AI 기능 통합 기회도 식별

G

## Generator

스프린트 단위

React + Vite + FastAPI + SQLite/PostgreSQL 스택. 한 번에 하나의 기능 구현, 자체 평가 후 QA에 전달, git으로 버전 관리

E

## Evaluator

Playwright MCP

실제 사용자처럼 앱과 상호작용. UI, API, DB 상태 테스트. 각 스프린트 전 **"스프린트 계약서"**로 완료 기준 협상

COMPARISON

# Solo Run vs Full Harness

RetroForge: 2D 레트로 게임 메이커 — Opus 4.5 기준

| 항목 | Solo Run | Full Harness |
|---|---|---|
| **소요 시간** | 20분 | 6시간 |
| **비용** | $9 | $200 |
| **기능 범위** | 기본 에디터 | 16개 기능, 10 스프린트 |
| **결과 품질** | 낭비된 레이아웃, 경직된 워크플로우, 깨진 게임 메카닉 | 스프라이트 애니메이션, 사운드, AI 생성, 공유 기능 |
| **핵심 차이** | **문제 표시 없음, UI 가이드 없음** | **일관된 시각 아이덴티티, 작동하는 게임** |

✓ Full Harness의 **Claude 통합**: 프롬프팅으로 게임 컴포넌트를 생성하여 워크플로우를 크게 가속

QA CHALLENGES

# QA 프로세스의 도전과 한계

### Evaluator의 초기 문제

1. 합법적 이슈를 식별하지만 **합리화** 하여 수용
2. **표면적 테스트** 만 수행, 엣지 케이스 미탐색
3. 기대 판단과의 차이를 기반으로 QA 프롬프트 반복 조정

### 조정 후에도 잔존하는 한계

- ⚠ 미세한 레이아웃 이슈
- ⚠ 비직관적으로 느껴지는 인터랙션
- ⚠ 중첩된 기능에 숨어있는 미발견 버그

> *"Evaluator는 상당한 튜닝이 필요했다. 기대 판단과의 차이를 기반으로 반복 업데이트해야 했다."*

04

# 하네스의 진화

Opus 4.6과 함께 불필요한 스캐폴딩 제거

KEY PRINCIPLE

# Opus 4.6으로 단순화

> *"하네스의 모든 컴포넌트는 모델이 혼자 할 수 없다는 가정을 인코딩한다. 그 가정은 스트레스 테스트할 가치가 있다."*

| 변화 | 이전 (Opus 4.5) | 이후 (Opus 4.6) |
|---|---|---|
| **스프린트 분해** | 필수 — 일관성 유지 불가 | 불필요 — 장시간 일관된 작업 |
| **Evaluator 역할** | 항상 필수 | 복잡한 태스크에만 가치 |
| **계획 수립** | 세밀한 스캐폴딩 필요 | 자체 계획 + 디버깅 가능 |
| **판단 기준** | **Evaluator의 필요성 = 태스크 복잡도 ÷ 모델 능력** | |

Opus 4.6 개선 사항: **향상된 계획 수립**, 지속적 에이전틱 태스크 수행, 대규모 코드베이스 안정 운영, 코드 리뷰/디버깅 강화, 긴 컨텍스트 검색 향상

CASE STUDY

# Browser DAW — $124.70으로 완성

"브라우저에서 Web Audio API를 활용한 완전한 기능의 DAW를 만들어라"

| 단계 | 시간 | 비용 |
|---|---|---|
| Planner | 4.7분 | $0.46 |
| Build 1 | 2h 07m | $71.08 |
| QA 1 | 8.8분 | $3.24 |
| Build 2 | 1h 02m | $36.89 |
| QA 2 | 6.8분 | $3.09 |
| Build 3 | 10.9분 | $5.88 |
| QA 3 | 9.6분 | $4.06 |
| **합계** | **~4시간** | **$124.70** |

**QA가 발견한 핵심 갭**

- ⚠ 클립 드래그/이동 불가
- ⚠ 악기 UI 패널 미구현 (신디사이저 노브, 드럼 패드)
- ⚠ 이펙트 시각화가 수치 슬라이더만 사용
- ⚠ 오디오 녹음이 스텁 상태

**최종 결과**

프롬프팅만으로 간단한 곡 제작 가능 — 템포, 키 설정, 멜로디, 드럼 트랙, 믹서 조정, 리버브 추가

05

# 핵심 교훈

모델이 발전해도 하네스의 공간은 축소되지 않는다 — 이동한다

KEY TAKEAWAYS

# 3가지 핵심 레슨

**01**

### 실험 우선

실제 문제에 대상 모델로 실험하고, 트레이스(실행 기록) 기반으로 성능을 튜닝하라. 이론이 아닌 관찰에서 시작

**02**

### 분해와 전문화

복잡한 태스크를 분해하고 특화된 에이전트를 각 측면에 적용하라. 생성과 평가의 분리가 핵심 레버

**03**

### 새 모델 출시 시 재검토

비핵심 컴포넌트를 제거하고, 더 큰 능력을 가능케 하는 새로운 조각을 추가하라. 하네스는 살아있는 시스템

> *"흥미로운 하네스 조합의 공간은 모델이 발전해도 축소되지 않는다. 대신 이동한다. AI 엔지니어의 흥미로운 작업은 다음 새로운 조합을 계속 찾는 것이다."*

ANTHROPIC

# 하네스는 축소되지 않는다
# 이동한다

Harness Design for Long-Running Application Development
Prithvi Rajasekaran · Anthropic Labs · 2026.03.24